

DE MORI VALENTINO Aurélie
FOUTEL Romane

Structure de données Historique Transactions

Master ICo - 2025/2026



**UNIVERSITÉ DE
MONTPELLIER**

Table des matières

I. Présentation du sujet.....	1
1. Présentation.....	1
2. Opérations considérées.....	1
II. Choix d'implémentation.....	2
1. Structures comparées - TreeSet vs TreeMap.....	2
2. Hypothèses de tri.....	2
III. Hypothèses de génération de données.....	2
1. Génération des données.....	2
2. Protocole Expérimental.....	3
IV. Scénarios de pourcentages.....	4
V. Résultats obtenus et interprétation.....	5
1. Comparaison des deux structures triées par identifiant.....	5
2. Comparaison des deux structures avec un TreeSet trié par date.....	6
3. Comparaison des différents scénarios effectués avec les deux structures.....	7
VI. Point sur les complexités théoriques.....	8
1. Opérations d'ajout et de suppression.....	8
2. Opération de recherche par identifiant.....	9
3. Parcours chronologique.....	9
4. Comptage par type.....	9
5. Complexité mémoire.....	9
VII. Conclusion.....	10
VIII. Pour aller plus loin.....	11
IX. Usage des LLM.....	12
1. Modèle utilisé.....	12
2. Evaluation de l'empreinte carbone.....	12
Annexe - Prompts.....	13

I. Présentation du sujet

1. Présentation

Le projet porte sur l'étude expérimentale du choix d'une structure de données dans un programme Java. Ici notre programme portera sur un **historique de transactions** qui nous permettra d'étudier deux structures différentes : **TreeSet** et **TreeMap**. Nous travaillerons avec deux conteneurs principaux : HistoriqueTreeSet et HistoriqueTreeMap utilisant chacun une structure de données différente et stockant des objets métiers Transaction.

Le travail consistera à comparer ces deux solutions du point de vue du **temps d'exécution**, de l'**occupation mémoire** et de déterminer quelle structure est la plus adaptée selon le scénario envisagé.

Aurélie s'est occupée de la TreeMap, de l'interface, de la mise en forme des résultats ainsi que d'une partie du Benchmark pendant que Romane a réalisé le TreeSet, la classe métier, les exceptions, la génération des données et l'autre partie du Benchmark.

2. Opérations considérées

Les opérations qui seront testées sont les suivantes :

- **Ajout** - addTransactions : Permet d'ajouter une transaction.
- **Annulation** - annulerTransaction : Permet d'annuler une transaction grâce à son identifiant. Dans le conteneur HistoriqueTreeSet, un itérateur est utilisé pour parcourir la liste et en même temps supprimer la transaction souhaitée (car le TreeSet est trié par ordre chronologique et non par identifiant).
- **Recherche par identifiant** - recherched : Permet de rechercher une transaction par son identifiant.
- **Parcours chronologique** - parcoursChronologique : Permet de parcourir les transactions par date et de renvoyer les transactions situées entre les deux dates passées en paramètre.
- **Comptage par type** - comptageType : Permet de compter le nombre de transactions du type passé en paramètre.

II. Choix d'implémentation

1. Structures comparées - TreeSet vs TreeMap

Nous nous intéressons à deux structures différentes :

- **TreeSet** : Cette structure stocke les objets Transaction de manière unique. L'ordre de l'arbre se fait grâce à la méthode compareTo de la classe métier Transaction.
- **TreeMap<String, Transaction>** : Cette structure fonctionne par paires Clé/Valeur. La clé est l'identifiant unique (String id) de la transaction tandis que l'objet Transaction constitue la valeur.

2. Hypothèses de tri

TreeSet (Tri Chronologique) : Dans notre implémentation, le TreeSet a un tri chronologique. En cas de dates identiques, un tri secondaire par identifiant est appliqué pour garantir l'unicité des éléments dans le Set. Le choix de trier le TreeSet par date repose sur l'usage principal d'un historique : la consultation chronologique.

TreeMap (Tri par Identifiant) : Contrairement au TreeSet, la TreeMap ignore le compareTo de l'objet Transaction. Elle est triée selon l'ordre lexicographique des clés (id) car une Map ne peut pas avoir deux fois la même clé : il est donc impossible de trier par date.

Comparer ses deux structures va mettre en évidence la rapidité d'accès à un objet Transaction par id avec la TreeMap et la facilité de parcours chronologique avec le TreeSet.

Un tri par identifiant des deux structures a également été testé (changement du tri de TreeSet en tri par identifiant) pour montrer que le TreeSet devient alors moins utile que la TreeMap car sa force réside dans sa capacité à maintenir un ordre naturel utile (ici, la date) sans coût supplémentaire.

III. Hypothèses de génération de données

1. Génération des données

Dans la classe GenerateurDonnees, nous avons implémenté une méthode qui permet de générer les données en grandes quantités. La génération se fait de la manière suivante :

- **Identifiants** : Chaque transaction se voit attribuer un identifiant unique suivant le format ID_ suivi de son index de création.
- **Types** : La répartition des types de transactions est fixée à 50% d'Achats et 50% de Ventes pour assurer l'homogénéité des données.

- **Date** : Les dates sont générées de manière incrémentale à partir de la date actuelle pour faciliter les tests de parcours chronologiques.
- **Montants** : Le montant de chaque transaction est calculé via la formule $(i \% 500) + 10$ pour obtenir des montants différents mais qui restent situés entre 10 et 509 euros.

Cette procédure de génération permet de confronter nos deux structures (TreeSet et TreeMap) à des volumes de données identiques.

2. Protocole Expérimental

Afin d'obtenir des mesures fiables, nous avons mis en place le protocole suivant :

- **Tailles testées** : Le benchmark est lancé sur des tailles variées (de 1000 à 50 000 éléments).
- **Répétitions** : Chaque mesure est répétée plusieurs fois (ici 20 fois) pour calculer une moyenne stable et neutraliser les fluctuations aléatoires du système.
- **Nombre d'opérations** : Pour chaque taille n , nous exécutons une série de nbOperations (ici 1000) pour simuler une activité réelle.
- **Isolations des tests** : À chaque répétition du test, l'historique est intégralement recréé à neuf pour éviter l'effet cumulatif des données des tests précédents.
- **Préchauffage** : Une phase de préchauffage (Warmup) est effectuée au démarrage du programme pour permettre l'optimisation JIT de la JVM.
- **Répartition paramétrable des opérations** : modulation facile des pourcentages attribués à chaque méthode du conteneur (ajout, annulation, recherche par identifiant, parcours chronologique et comptage par type) pour simuler différents scénarios.

Nous collectons les données suivantes :

- **Temps d'exécution** : La durée des opérations est calculée par la différence entre deux captures de `System.nanoTime()` effectuées avant et après l'exécution du scénario de test ou de l'opération.
- **Occupation mémoire** : L'occupation RAM est évaluée en soustrayant la mémoire libre de la mémoire totale via `Runtime.getRuntime()` après avoir utilisé le Garbage Collector pour stabiliser les mesures.

Pour obtenir les mesures les plus précises possibles, nous avons procédé en deux étapes. D'abord, nous avons réalisé des tests unitaires en répétant 10 000 fois les opérations d'ajout, de suppression et de recherche par identifiant. Cela nous a permis de calculer une moyenne fiable pour le temps et la mémoire d'une seule

opération en évitant les micro-variations du système. Pour le parcours et le comptage, nous avons mesuré le temps global sur toute la structure pour ensuite en déduire le coût d'une opération.

Après cela, nous avons lancé les différents scénarios pour générer les graphiques pour comparer les deux structures et vérifier visuellement si les résultats suivent les courbes théoriques de chaque structure.

IV. Scénarios de pourcentages

Nous nous sommes intéressés à trois scénarios différents :

- **Scénario 1 - Gestion de compte bancaire** : Ce scénario simule l'activité d'un conseiller bancaire gérant le traitement des transactions. Chaque transaction (virement, dépôt, prélèvement) doit être tracée, vérifiée ou parfois annulée. Ce scénario privilégie donc les opérations d'ajout, d'annulation et de recherche par identifiant.

Répartition des opérations : 35% d'ajouts, 25% d'annulations, 30% de recherche par identifiant, 5% de parcours chronologique et 5% de comptage par type.

- **Scénario 2 - Consultation Client** : Ce scénario simule l'usage d'une application mobile bancaire par un particulier. L'utilisateur se connecte principalement pour consulter l'historique de ses dernières opérations et vérifier la répartition de ses dépenses. Ce scénario se concentre donc principalement sur l'opération de parcours chronologique.

Répartition des opérations : 5% d'ajouts, 5% d'annulations, 10% de recherche par identifiant, 70% de parcours chronologique et 10% de comptage par type.

- **Scénario 3 - Trading** : Ce scénario simule un système de trading. L'objectif est la collecte intensive de transactions boursières et l'analyse des tendances en temps réel. Ce scénario aura donc un fort pourcentage d'ajouts et de comptage par type.

Répartition des opérations : 45% d'ajouts, 5% d'annulations, 5% de recherche par identifiant, 10% de parcours chronologique et 35% de comptage par type.

V. Résultats obtenus et interprétation

1. Comparaison des deux structures triées par identifiant

=== TABLEAU TEMPS CALCUL DES OPERATIONS (ms) ===			=== TABLEAU CALCUL MÉMOIRE DES OPERATIONS (octets) ===		
Nom du Test	TreeSet (ms)	TreeMap (ms)	Nom de l'opération	TreeSet (oct)	TreeMap (oct)
Génération des données	90,8829	9,0440	Génération des données	7398312,0000	1476864,0000
Ajout d'une transaction	0,0091	0,0009	Ajout d'une transaction	739,8312	147,6864
Comptage par type	10,7541	1,7100	Comptage par type	449,9922	41,1875
Recherche par identifiant	0,0008	0,0006	Recherche par identifiant	87,9824	50,6208
Parcours chronologique	10,4371	13,6767	Parcours chronologique	181,8359	328,3125
Suppression d'une transaction	0,0043	0,0004	Suppression d'une transaction	109,6766	50,6259

Figure 1 : Tableaux des opérations avec TreeSet trié par identifiant

Analyse des performances temporelles (Tableau de gauche de la Figure 1)

L'analyse des temps d'exécution met en évidence une supériorité de la TreeMap du point de vue temporel :

- **Génération de données** : La TreeMap est 10 fois plus rapide que le TreeSet.
- **Recherche par identifiant** : Les deux structures affichent des performances très proches (0,0008 ms pour le TreeSet contre 0,0006 ms pour la TreeMap). Cela prouve que l'implémentation du tri par identifiant permet au TreeSet de bénéficier de la recherche logarithmique $O(\log n)$ comme la TreeMap.
- **Parcours chronologique** : On observe des temps élevés pour les deux structures (10,43 ms et 13,67 ms). Le tri étant effectué sur l'identifiant et non sur la date, le TreeSet perd son efficacité naturelle pour cette opération et doit réaliser un parcours complet $O(n)$ pour filtrer les transactions.

Analyse de l'empreinte mémoire (Tableau de droite de la Figure 1)

Les résultats en octets montrent une consommation de ressources plus élevée pour le TreeSet :

- **Coût de stockage global** : La génération des données consomme 7,2 Mo pour le TreeSet contre seulement 1,4 Mo pour la TreeMap. Le TreeSet occupe donc environ 5 fois plus d'espace mémoire que la TreeMap pour stocker le même volume d'informations.
- **Opérations unitaires** : Pour chaque ajout ou suppression, le TreeSet mobilise systématiquement plus de mémoire (739,8 octets pour un ajout contre 147,6 octets pour la TreeMap).

Le TreeSet est en réalité une TreeMap déguisée qui stocke les données comme des clés et leur associe automatiquement un objet inutile pour fonctionner. Il est donc plus lent car il doit effectuer des manipulations supplémentaires pour gérer cet objet

inutile à chaque insertion alors que la TreeMap travaille directement avec la clé et la valeur fournie.

Ces résultats démontrent que l'utilisation d'un TreeSet trié par identifiant est inefficace par rapport à une TreeMap car la force du TreeSet réside normalement dans sa capacité à maintenir un ordre naturel (comme la chronologie). Ici, le tri par identifiant rend le parcours chronologique coûteux $O(n)$ sans pour autant égaler la légèreté de la TreeMap pour la gestion des identifiants. C'est pourquoi dans la suite du projet, le TreeSet sera trié par ordre chronologique.

2. Comparaison des deux structures avec un TreeSet trié par date

=== TABLEAU TEMPS CALCUL DES OPERATIONS (ms) ===			=== TABLEAU CALCUL MÉMOIRE DES OPERATIONS (octets) ===		
Nom du Test	TreeSet (ms)	TreeMap (ms)	Nom de l'opération	TreeSet (oct)	TreeMap (oct)
Génération des données	160,0508	10,3045	Génération des données	2140112,0000	1553712,0000
Ajout d'une transaction	0,0160	0,0010	Ajout d'une transaction	214,0112	155,3712
Comptage par type	5,8830	1,6357	Comptage par type	234,9141	75,8594
Recherche par identifiant	0,0519	0,0004	Recherche par identifiant	341,3656	55,1808
Parcours chronologique	1,6643	17,9924	Parcours chronologique	169,8594	335,4375
Suppression d'une transaction	0,0004	0,0007	Suppression d'une transaction	84,8141	56,8833

Figure 2 : Tableaux des opérations avec TreeSet trié par date

Analyse des Performances Temporelles (Tableau de gauche de la Figure 2)

Le choix de la structure de données entraîne un compromis majeur entre la vitesse de recherche et la vitesse de parcours :

- **Génération des données** : La TreeMap est nettement plus rapide lors de la génération initiale (10,30 ms contre 160,05 ms) et pour l'ajout unitaire (0,0010 ms contre 0,0160 ms).
- **Recherche par identifiant** : La recherche par identifiant est quasi instantanée sur la TreeMap (0,0004 ms) grâce à l'indexation par clé tandis que le TreeSet est beaucoup plus lent (0,0519 ms) car il est maintenant trié par date et n'est donc pas optimisé pour les recherches par identifiant.
- **Parcours chronologique** : Le TreeSet effectue le parcours chronologique en 1,6643 ms, alors que la TreeMap nécessite 17,9924 ms. Puisque le TreeSet est déjà trié par date, il accède aux données beaucoup plus rapidement.

Analyse de la Consommation Mémoire (Tableau de droite de la Figure 2)

L'écart de mémoire entre les deux structures est flagrant et systématique :

- **Génération des données** : Le TreeSet occupe 2,04 Mo contre seulement 1,48 Mo pour la TreeMap.

- **Surcoût par ajout** : L'ajout d'une transaction utilise 214 octets dans le TreeSet contre environ 155 octets dans la TreeMap.

Ce surcoût confirme l'utilisation par le TreeSet d'une TreeMap sous-jacente où chaque entrée est associée à un objet fantôme augmentant donc l'empreinte mémoire pour chaque nœud de l'arbre.

Cette comparaison démontre que la TreeMap est la structure la plus polyvalente et la plus légère en mémoire. Cependant, le TreeSet trié par date reste indispensable si l'application doit effectuer des affichages chronologiques fréquents car il est 10 fois plus performant sur cette opération spécifique malgré sa lourdeur mémoire et sa lenteur à l'insertion.

3. Comparaison des différents scénarios effectués avec les deux structures

Scénario 1 - Gestion compte bancaire (Figure 3)

Dans ce scénario qui privilégie les ajouts, les annulations et les recherches par identifiant, l'écart est massif :

Performance : La **TreeMap** (bleu) largement plus rapide que le **TreeSet** (rouge).

Analyse : La pente du TreeSet s'envole à mesure que la taille augmente, atteignant plus de 500 ms pour 50 000 éléments. À l'inverse, la TreeMap reste stable sous la barre des 100 ms. Cela confirme que pour une gestion basée sur des identifiants uniques, la structure de TreeMap est bien plus efficace que le TreeSet.

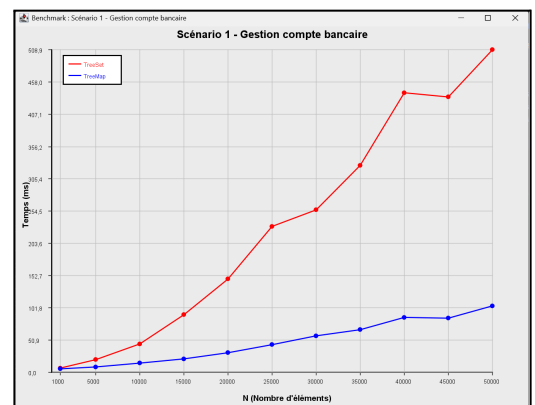


Figure 3 : Graphe scénario Gestion compte bancaire

Scénario 2 - Consultation client (Figure 4)

Ce scénario privilégie le parcours chronologique et fait donc ressortir la performance du TreeSet :

Performance : Le **TreeSet** (rouge) est beaucoup plus rapide que la **TreeMap** (bleu).

Analyse : Le TreeSet maintient un temps d'exécution bas (environ 230 ms au maximum), tandis que la TreeMap explose pour atteindre près de 750 ms. Le TreeSet étant trié par date dès l'insertion, il traite les requêtes chronologiques de

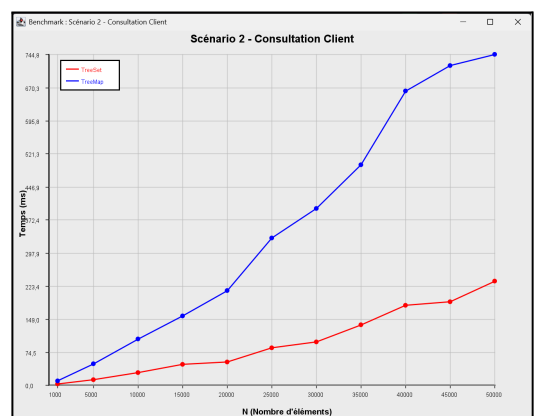


Figure 4 : Graphe scénario Consultation client

manière quasi directe, contrairement à la TreeMap qui doit fournir un effort de tri ou de parcours supplémentaire.

Scénario 3 - Trading (Figure 5)

Le scénario de trading présente un profil hybride avec des opérations d'ajouts et de comptages par type :

Performance : Les deux courbes sont quasiment identiques et se croisent régulièrement.

Analyse : Jusqu'à 25 000 éléments, les deux structures se valent. Au-delà, on observe une légère instabilité due aux opérations de comptage qui sollicitent les deux arbres de manière similaire. Pour ce type d'utilisation, le choix de la structure a un impact négligeable sur le temps d'exécution final, les deux plafonnant autour de 400 ms pour 50 000 éléments.

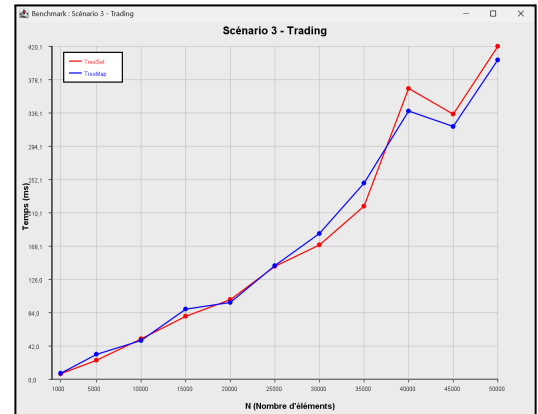


Figure 5 : Graphe scénario Trading

Scénario	Opérations Dominantes	Vainqueur (Temps)	Vainqueur (Mémoire)	Justification Technique
Gestion de compte bancaire	Ajouts, Annulations, Recherche par ID	TreeMap	TreeMap	La TreeMap indexe par id ce qui rend la recherche par id très rapide. Le TreeSet doit parcourir un arbre trié par date pour trouver un id.
Consultation Client	Parcours chronologique	TreeSet	TreeMap	Le TreeSet est trié par date à l'insertion : le parcours est direct (1,66 ms). La TreeMap doit parcourir toute sa structure (17,99 ms).
Trading	Ajouts, Comptage par type	Identique	TreeMap	Les deux structures ont une complexité en $O(\log(n))$ pour l'ajout. L'impact du type de tri est ici négligeable sur le temps, mais la TreeMap reste plus légère.

Figure 6 : Tableau récapitulatif des scénarios

Nous pouvons donc en conclure que le TreeSet est plus efficace pour les tri par dates et la TreeMap pour les gestions par identifiant mais que de manière globale la TreeMap est plus légère en occupation mémoire.

VI. Point sur les complexités théoriques

1. Opérations d'ajout et de suppression

Les structures TreeMap et TreeSet sont implémentées sous forme d'arbre binaire de recherche (ordonné) équilibré. Si n est le nombre de nœuds, la profondeur ne dépasse jamais $2 \log_2(n)$.

Pour ajouter ou supprimer, l'algorithme descend le long d'une branche de l'arbre.

Comme on divise l'espace de recherche par deux à chaque nœud, le nombre d'opérations est proportionnel à la hauteur de l'arbre, soit $O(\log(n))$.

2. Opération de recherche par identifiant

La complexité de cette opération dépend de si l'identifiant est la clé de tri de la structure ou un simple attribut.

Comme notre TreeMap est triée par identifiant, sa complexité sera de $O(\log(n))$.

Le TreeSet, lui, est trié par date. Il n'y a donc aucun ordre logique pour les identifiants. Pour trouver un identifiant spécifique, l'algorithme est contraint de visiter chaque nœud un par un jusqu'à trouver le bon et effectue donc une recherche linéaire. Il aura donc une complexité de $O(n)$.

3. Parcours chronologique

Dans le TreeSet, les éléments sont déjà triés par date. Il suffit donc de parcourir l'arbre pour obtenir la liste ordonnée ce qui prend un temps proportionnel au nombre d'éléments soit $O(n)$.

Dans le TreeMap, puisque les données sont rangées par id, elles ne sont pas dans l'ordre chronologique. Il faut d'abord extraire les n éléments $O(n)$ puis appliquer un algorithme de tri dont la complexité est $O(n \log(n))$.

4. Comptage par type

Aucune des deux structures n'est indexée sur l'attribut type. Pour compter le nombre d'occurrences d'un type, le programme doit donc obligatoirement inspecter chaque transaction. Nous avons donc un temps de calcul qui croît linéairement avec la taille des données. Les deux structures auront donc une complexité de $O(n)$.

5. Complexité mémoire

La complexité en place pour un ensemble est de "nombre d'éléments x taille d'un élément" soit $O(n)$.

En Java, le TreeSet utilise en réalité une instance de TreeMap en interne pour stocker ses éléments. Chaque entrée dans un TreeSet crée un objet "Clé-Valeur" (où la valeur est une constante inutile) ce qui consomme plus de mémoire qu'une simple association directe. Le TreeSet aura donc une complexité en espace de $O(n)$ + surcoût.

Nous obtenons donc le tableau récapitulatif suivant :

Opération	TreeMap (par ID)	TreeSet (par Date)
Ajout / Suppression	$O(\log(n))$	$O(\log(n))$
Recherche par id	$O(\log(n))$	$O(n)$
Parcours chronologique	$O(n\log(n))$	$O(n)$
Comptage par type	$O(n)$	$O(n)$
Mémoire	$O(n)$	$O(n) + \text{surcoût}$

Figure 7 : Tableau récapitulatif des complexités théoriques

VII. Conclusion

La TreeMap est la structure la plus polyvalente et doit être le choix par défaut dans la majorité des architectures logicielles. Elle est plus efficace en mémoire, et la recherche et la suppression par identifiant sont quasi instantanées même sur des grands volumes de données.

Elle est idéale pour les systèmes transactionnels, les outils de gestion de back-office ou toute base de données nécessitant des modifications et annulations fréquentes d'enregistrements spécifiques via leur ID.

De son côté, le TreeSet est une structure spécialisée dont l'usage doit être dicté par un besoin métier spécifique lié à l'ordre des éléments. Son point fort réside dans le parcours des données et donc, dans notre cas, le parcours chronologique. Son efficacité est contrebalancée par une recherche par identifiant plus lente $O(n)$ et une consommation mémoire plus élevée.

Son contexte idéal serait les applications qui nécessitent principalement des affichages chronologiques fréquents comme des relevés clients ou encore les flux d'actualités : C'est la structure privilégiée dès que l'utilisateur consulte les données de la plus récente à la plus ancienne.

En conclusion pour notre projet, la TreeMap est préférable si l'application nécessite une manipulation intensive et rapide des données. Et si l'application est avant tout utilisée pour consulter les Transactions, le TreeSet devient alors la meilleure option.

VIII. Pour aller plus loin

Dans une application réelle, les deux pourraient être utilisés simultanément pour bénéficier de leurs avantages respectifs.

Les transactions seraient stockées dans une TreeMap pour que l'application puisse modifier, annuler ou vérifier une transaction instantanément. Le TreeSet, lui, contiendrait des références vers ces mêmes transactions et ne servirait qu'à l'affichage chronologique.

Cela permettrait d'avoir la recherche ultra-rapide par identifiant et l'affichage chronologique instantané mais cela doublerait la consommation mémoire car cela nécessiterait de mettre à jour deux structures à chaque ajout.

IX. Usage des LLM

1. Modèle utilisé

Le modèle a été sollicité tout au long du projet pour l'amélioration de code et débogage, la conception de scénarios, l'analyse de données et l'aide à la rédaction.

Nom du modèle : Gemini 3 Flash.

Version : Web (Tier Gratuit).

Développeur : Google.

Période d'utilisation : Avril 2026.

2. Evaluation de l'empreinte carbone

Pour réaliser ce calcul, nous supposons que nous avons réalisé environ 300 prompts avec Gemini.

Selon les données publiées par Google et analysées par le cabinet de conseil carbone Sami, une requête médiane sur Gemini émet environ 0,03 g de CO₂ émis. Pour l'ensemble du projet, l'empreinte carbone s'élève donc à environ 9 g de CO₂ émis. Cela correspond à environ un trajet de moins de 60 mètres en voiture thermique.

Ce faible résultat s'explique d'abord par l'utilisation du modèle Flash de Gemini, une version allégée de l'IA qui demande beaucoup moins de puissance de calcul et d'énergie pour répondre. Il reflète aussi la comptabilité de Google qui soustrait ses achats d'énergie verte de sa consommation réelle ce qui fait chuter artificiellement le bilan carbone final.

Source (trouvée par Adrien Devezeaud) : *Sami.eco (2026), "L'IA en entreprise : quel impact carbone et comment le maîtriser ?"*

lien : <https://www.sami.eco/blog/ia-impact-carbone-entreprise>

Annexe - Prompts

Voici un extrait des prompts qui ont pu être réalisés durant le projet :

Le 04/04 - Conception des scénarios

Présente moi 3 scénarios différents pour des Historiques de Transactions avec une répartition des opérations différentes à chaque fois :

a% d'ajouts, b% d'annulations, c% de recherches, d% de parcours par date, e% de comptages par type

Le 04/04 - Amélioration de code

Voici ma méthode de suppression par itérateur. Est-ce qu'il existe une meilleure solution qui nous donnerait une meilleure complexité temporelle ?

Le 29/04 - Analyse des données

Voici mes mesures de temps pour le scénario 2 qui privilégie l'opération de parcours chronologique. Peux tu me les analyser ?

Le 29/04 - Aide à l'analyse

Explique-moi pourquoi le TreeSet consomme plus de mémoire qu'une TreeMap.

Le 29/04 - Aide à la rédaction

Aide moi à rédiger une conclusion en précisant les points forts et faibles de chaque structure (TreeSet et TreeMap) et en donnant les contextes où chaque structure est préférable.